Yul & Assembly

By Code Eater

Yul

Yul is an **intermediate language** that can be used within Solidity or as a stand-alone language. It is particularly useful for writing highly optimized smart contracts. Yul is designed to be simple, with a focus on performance and minimalism.

Yul itself does not have explicit data types like high-level programming languages such as Solidity or JavaScript. Yul is designed as a minimalistic intermediate language that operates directly with the Ethereum Virtual Machine (EVM).

What solidity cannot do…

```
contract SimpleStorage {
    uint256 private value;

    // Store a value (not at a specific slot)
    function storeValue(uint256 newValue) public {    22498 gas
        value = newValue;
    }

    // Retrieve the stored value
    function retrieveValue() public view returns (uint256) {
        return value;
    }
}
```

By default slot 0 will be used in this case for storage.

```
contract AssemblyStorage {
    // Store a value at a specific storage slot
    function storeValue(uint256 slot, uint256 value) public {    infinite gas
        assembly {
            sstore(slot, value)
        }
    }

    // Retrieve a value from a specific storage slot
    function retrieveValue(uint256 slot) public view returns (uint256) {
        uint256 value;
        assembly {
            value := sload(slot)
        }
        return value;
    }
}
```

Flexibility to choose slots.

Another Difference…

- Yul involves no check you are on your own. While in solidity there are a lot of checks.

Note - Yul gives you more control than Solidity but this comes with a price. You have to be very careful while writing Yul as you there are no security checks. As a developer it's your responsibility to write the correct code.

Yul Instructions

Instruction			Explanation
stop()	-	F	stop execution, identical to return(0, 0)
add(x, y)		F	x + y
sub(x, y)		F	x - y
mul(x, y)		F	x * y
div(x, y)		F	x / y or 0 if y == 0
sdiv(x, y)		F	x / y, for signed numbers in two's complement, 0 if y == 0
mod(x, y)		F	x % y, 0 if y == 0
smod(x, y)		F	x % y, for signed numbers in two's complement, 0 if y == 0
exp(x, y)		F	x to the power of y
not(x)		F	bitwise "not" of x (every bit of x is negated)
lt(x, y)		F	1 if x < y, 0 otherwise
gt(x, y)		F	1 if x > y, 0 otherwise

Code Eater

Click For More

Yul in pure form

```
{
    // Declare variables without specifying types
    let a
    let b

    // Assign values to variables
    a := 10
    b := 20

    // Perform arithmetic operation (implicitly handled by EVM)
    let c := add(a, b)

    // Store result in memory
    mstore(0x00, c)

    // Return memory position and size
    return(0x00, 0x20)
}
```

Code Eater

Yul

```
contract InlineAssemblyExample {
    function add(uint256 a, uint256 b) public pure returns (uint256) {
        uint256 result;
        assembly {
            result := add(a, b)
        }
        return result;
    }
}
```

Curly Braces

Use Of :

No Semicolon

Storage in Yul

```
contract StorageExample {
    // Slot 0 is used for storing the number
    uint256 private storedNumber=10;

    // Function to store a number in storage using Yul
    function storeNumber(uint256 num) public {    22512 gas
        assembly {
            // Store the number in slot 0
            sstore(0, num)
        }
    }

    // Function to retrieve the stored number from storage using Yul
    function retrieveNumber() public view returns (uint256) {    2428 gas
        uint num;
        assembly {
            // Load the number from slot 0
            num := sload(0)
        }
        return num;
    }
}
```

Variable Packing

Variable packing in Solidity is a technique used to optimize the storage of state variables in a contract. Solidity stores state variables in slots of 32 bytes (256 bits). When multiple variables can fit into a single 32-byte slot, Solidity will pack them together to use storage more efficiently.

```
contract Demo {
    uint8 public a = 10; //slot 0
    uint8 public b = 0xff; //slot 0
    bool public c = true; //slot 0

    // Function to read the contents of a storage slot
    function getSlotValue(uint slot) public view returns (bytes32) {
        bytes32 value;
        assembly {
            value := sload(slot)
        }
        return value;
    }
}
```

```
//0x0000000000000000000000000000000000000000000000000000000000001ff0a - slot 0
```

Reading A Slot (Bit Shifting)

```
contract OffsetAndShfiting {
    uint128 public A = 22;
    uint96 public B = 15;
    uint16 public C = 8;
    uint8 public D = 1;

    function readValueBySlot(uint256 slot) external view returns (bytes32 value) {
        assembly {
            value := sload(slot)
        }
    }

    function getOffsetOfC() external pure returns (uint256 slot, uint256 offset) {
        assembly {
            slot := C.slot
            offset := C.offset//offset tell the position of C.
                                //In short go 28 bytes to the left to find the vlaue of C
        }
    }
    function readValueOfC() external view returns (uint256 c) {    2533 gas
        assembly {
            let value := sload(C.slot)
            // C.offset = 28
            let shifted := shr(mul(C.offset, 8), value) //shifting right by 28*8 bits
            // 0x0000000000000000000000000000000000000000000000000000000000010008
            // `and` operation will be applied
            // 0x000000000000000000000000000000000000000000000000000000000000ffff
            c := and(0xffff, shifted)
        }
    }
```

Code Eater

Writing A Slot (Bit Shifting)

```
function writeToC(uint16 newC) external {    24708 gas
    assembly {
        // newC = 0x000000000000000000000000000000000000000000000000000000000000000a
        let slotValue := sload(C.slot)
        // slotValue =  0x00010008000000000000000000000000f000000000000000000000000000000016
        let clearedC := and(
            slotValue,
            0xffff0000ffffffffffffffffffffffffffffffffffffffffffffffffffffffff
        )
        // mask      =  0xffff0000ffffffffffffffffffffffffffffffffffffffffffffffffffffffff
        // slotValue =  0x00010008000000000000000000000000f000000000000000000000000000000016
        // clearedC =   0x00010000000000000000000000000000f000000000000000000000000000000016
        let shiftedNewC := shl(mul(C.offset, 8), newC)
        // shiftedNewC = 0x0000000a000000000000000000000000000000000000000000000000000000000000
        let newSlotValue := or(shiftedNewC, clearedC)
        // shiftedNewC = 0x0000000a000000000000000000000000000000000000000000000000000000000000
        // clearedC    = 0x00010000000000000000000000000000f000000000000000000000000000000016
        // newVal      = 0x0001000a000000000000000000000000f000000000000000000000000000000016
        sstore(C.slot, newSlotValue)
    }
}
```

Practical Example

```
function _revert(bytes memory returndata) private pure {
    // Look for revert reason and bubble it up if present
    if (returndata.length > 0) {
        // The easiest way to bubble the revert reason is using memory via assembly
        assembly ("memory-safe") {
            let returndata_size := mload(returndata)
            revert(add(32, returndata), returndata_size)
        }
    } else {
        revert Errors.FailedCall();
    }
}
```

Solidity has limitations when dealing with memory manipulation, especially when working with raw data like returndata. Using inline assembly allows for precise memory operations, enabling OpenZeppelin to:

- Read the length of the returndata.
- Revert with only the relevant portion of the data (skipping the first 32 bytes, which contain the length field).

Code Eater

Memory in Yul

```
contract MemoryExample {
    function storeAndLoad(uint256 x) public pure returns (uint256) {
        uint256 result;
        assembly {
            // Store x at memory location 0x80
            mstore(0x80, x)
            // Load the value from memory location 0x80
            result := mload(0x80)
        }
        return result;
    }
}
```

Control Flow in Yul

```
contract ControlFlowExample {
    function conditional(uint256 x) public pure returns (uint256) {
        uint256 result;
        assembly {
            // If x is less than 10
            if lt(x, 10) {
                // Set result to 1
                result := 1
            }
            // Else case, if x is not less than 10
            if iszero(lt(x, 10)) {
                // Set result to 2
                result := 2
            }
        }
        return result;
    }
}
```

Code Eater

Loop

```
contract LoopExample {
    function sum(uint256 n) public pure returns (uint256) {
        uint256 result;
        assembly {
            let i := 0
            for { } lt(i, n) { i := add(i, 1) } {
                result := add(result, i)
            }
        }
        return result;
    }
}
```

Functions in Yul

```
contract FunctionExample {
    function factorial(uint256 n) public pure returns (uint256) {
        uint256 result;
        assembly {
            // Define the recursive factorial function
            function fact(x) -> y {
                switch x
                case 0 {
                    y := 1
                }
                default {
                    y := mul(x, fact(sub(x, 1)))
                }
            }
            // Call the factorial function with input n
            result := fact(n)
        }
        return result;
    }
}
```

Code Eater

Working with Storage

```
contract StorageExample {
    uint256 public data;

    function storeData(uint256 x) public {        22512 gas
        assembly {
            sstore(data.slot, x)
        }
    }

    function loadData() public view returns (uint256) {
        uint256 result;
        assembly {
            result := sload(data.slot)
        }
        return result;
    }
}
```

Code Eater

String in Yul

```
contract StringHandlingExample {

    function getFirstCharacter(string memory input) public pure returns (bytes1) {
        bytes memory inputBytes = bytes(input);
        bytes1 firstChar;

        assembly {
            let inputPtr := add(inputBytes, 0x20)
            firstChar := mload(inputPtr)
        }

        return firstChar;
    }

}
```

Handling strings in Yul can be challenging because Yul is a low-level language designed for efficiency and simplicity, and it doesn't have native support for high-level data types like strings. Strings in Solidity are dynamically sized arrays of bytes, which means you'll need to handle them as such in Yul.Strings in Solidity are dynamically sized byte arrays stored in memory. Assigning a string value directly in inline assembly requires careful memory management.

Code Eater

Dealing With String

```
contract StringExample {
    function getString() external pure returns (string memory) {
        string memory str = "";

        assembly {
            // Return the string myString
            str:="hello world"
        }
        return str;
    }
}
```

When you do str:="hello world" it does not make any sense. This is because strings and byte arrays in Yul are represented as pointers to memory locations and their lengths. They require explicit memory management operations like mstore to allocate and set their values correctly.

```
contract StringReturnExample {
    function getString() public pure returns (string memory) {   infinite ga
        string memory result;

        assembly {
            // Allocate memory for a string in Solidity format (32 bytes for
            result := mload(0x40)              // Set result to point to the
            mstore(result, 5)                  // Store the length of the str
            mstore(add(result, 0x20), 0x48656c6c6f000000000000000000000000

            // Update free memory pointer
            mstore(0x40, add(result, 0x40))    // Move the free memory pointe

        }

        return result; // Return the memory pointer containing the string
    }
}
```

Correct way to deal with string data type.

Bytes32 in Yul

```
contract StringExample {
    function getString() external pure returns (bytes32) {
        bytes32 str = "";

        assembly {
            str:="hello world"
        }
        return str;
    }
}
```

String in Yul

```
contract StringHandlingExample {
    function storeString(string memory input) public pure returns (string memory) {
        bytes memory inputBytes = bytes(input);
        uint256 length = inputBytes.length;

        // Allocate memory for the output string
        string memory output = new string(length);
        bytes memory outputBytes;

        assembly {
            // Get the memory locations of the input and output strings
            let inputPtr := add(inputBytes, 0x20)
            outputBytes := add(output, 0x20)

            // Copy the input string to the output string
            for { let i := 0 } lt(i, length) { i := add(i, 1) } {
                let char := mload(add(inputPtr, i))
                mstore(add(outputBytes, i), char)
            }
        }

        return output;
    }
}
```

Code Eater

Building a Simple Token

```
contract YulToken {
    string public name = "YulToken";
    string public symbol = "YUL";
    uint8 public decimals = 18;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    function transfer(address to, uint256 value) public returns (bool success) {
        assembly {
            let from := caller()
            let fromBalance := sload(add(balanceOf.slot, from))
            if lt(fromBalance, value) { revert(0, 0) }
            sstore(add(balanceOf.slot, from), sub(fromBalance, value))
            sstore(add(balanceOf.slot, to), add(sload(add(balanceOf.slot, to)), value))
            success := 1
        }
    }
}
```

Code Eater

Assignment

Find if a number is prime or not in Yul.

🔹 Why Do We Use Yul in Solidity?

Yul is an intermediate-level assembly language used in Solidity to optimize gas costs and improve efficiency.
It allows direct interaction with the Ethereum Virtual Machine (EVM) while bypassing Solidity's extra overhead.

🔹 Key Reasons to Use Yul

1️⃣ Gas Optimization (Cheaper Transactions)

- **Solidity adds extra instructions for safety** (like bounds checking, automatic zeroing, etc.).
- **Yul eliminates unnecessary steps**, reducing the gas cost of operations.

📌 **Example:**
Setting a state variable **without Yul** vs. **with Yul**:

solidity Copy Edit

```solidity
// Without Yul (Higher Gas)
function setNum(uint256 _num) public {
    num = _num;  // Uses extra Solidity instructions
}

// With Yul (Lower Gas)
function setNum(uint256 _num) public {
    assembly {
        sstore(num.slot, _num) // Direct storage write, less gas
    }
}
```

✅ **Benefit:** Lower gas fees (important for large-scale smart contracts).

2️⃣ Direct Storage & Memory Access

- Solidity functions like `storage`, `memory`, and `calldata` **consume more gas** due to additional safety checks.
- Yul **accesses storage/memory directly**, making operations cheaper.

📌 **Example:**

Retrieving a state variable using Solidity vs. Yul:

solidity Copy Edit

```solidity
// Without Yul (Higher Gas)
function getNum() public view returns (uint256) {
    return num;  // Uses extra Solidity instructions
}

// With Yul (Lower Gas)
function getNum() public view returns (uint256 result) {
    assembly {
        result := sload(num.slot) // Directly loads storage slot
    }
}
```

✅ **Benefit:** Faster execution and lower gas consumption.

3️⃣ Fine-Grained Control Over EVM

- Solidity abstracts **low-level EVM details**, sometimes making contracts inefficient.
- Yul allows **manual control over stack, memory, and storage.**

📌 **Example:**
Using `call()` in Solidity vs. Yul:

```solidity
// Solidity call() - Extra overhead
(bool success, bytes memory data) = address(target).call{value: msg.value}(payload);

// Yul call() - More efficient
assembly {
    let success := call(gas(), target, callvalue(), add(payload, 32), mload(payload), 0, 0)
}
```

✅ **Benefit:** More control over raw EVM operations.

4️⃣ Avoiding Solidity Compiler Bugs

- The Solidity compiler **sometimes introduces inefficiencies or unexpected behavior.**
- Yul **ensures lower-level control**, avoiding Solidity-specific bugs.

📌 **Example:** Some Solidity versions have inefficient `for` loops that can be optimized using Yul.

5️⃣ Efficient Looping & Arithmetic Operations

- Solidity loops involve **bounds checks, unnecessary variable resets.**
- Yul eliminates these unnecessary operations.

📌 **Example:**

Looping in Solidity vs. Yul:

solidity Copy Edit

```solidity
// Solidity loop - Extra safety checks
for (uint256 i = 0; i < array.length; i++) {
    sum += array[i];
}

// Yul loop - No extra checks (More Gas Efficient)
assembly {
    let len := mload(array)
    for { let i := 0 } lt(i, len) { i := add(i, 1) } {
        sum := add(sum, mload(add(array, mul(i, 32))))
    }
}
```

✅ **Benefit:** Less gas used in large loops.

🔹 When Should You Use Yul?

Use Yul When...	Avoid Yul When...
You need **gas optimization** (storage, loops, calls).	You need **safe, high-level logic.**
You want **manual control over storage & memory.**	You are writing a simple contract.
You are interacting with **EVM opcodes** directly.	You need readability & maintainability.
You need **custom cryptographic operations.**	You are dealing with complex business logic.

🔹 Conclusion

✅ Yul is used for gas savings, direct EVM control, and efficiency.
✅ It's powerful but requires expertise (avoid for complex logic).
✅ Best used for low-level storage access, looping optimizations, and interacting with the EVM.

🚀 Use Yul when necessary—but don't overuse it!

```
contract OffsetAndShfiting {
    uint128 public A = 22;
    uint96 public B = 15;
    uint16 public C = 8;
    uint8 public D = 1;

    function readValueBySlot(uint256 slot) external view returns (bytes32
value) {
        assembly {
            value := sload(slot)
        }
    }

    function getOffsetOfC() external pure returns (uint256 slot, uint256
offset) {
        assembly {
            slot := C.slot
            offset := C.offset//offset tell the position of C.
                                //In short go 28 bytes to the left to find the
vlaue of C
        }
    }
    function readValueOfC() external view returns (uint16 c) {
```

```
        assembly {
            let value := sload(C.slot)
            // C.offset = 28
            let shifted := shr(mul(C.offset, 8), value) //shifting right by
28*8 bits
            //
0x0000000000000000000000000000000000000000000000000000000000010008
            // `and` operation will be applied
            //
0x000000000000000000000000000000000000000000000000000000000000ffff
            c := and(0xffff, shifted)
        }
    }
}
```

Convert to **binary**:

yaml

```yaml
0xFFFF = 1111 1111 1111 1111  (16-bit)
0x0008 = 0000 0000 0000 1000  (16-bit)
```

Perform **bitwise AND** (`&`):

yaml

```yaml
   1111 1111 1111 1111
&  0000 0000 0000 1000
--------------------
   0000 0000 0000 1000  (Result)
```

Convert back to **hex**:

yaml

```yaml
0000 0000 0000 1000  =  0x0008
```

String in Yul

```
pragma solidity ^0.8.0;

contract StringHandlingExample {
    function getFirstCharacter(string memory input) public pure returns
(bytes1) {
        bytes memory inputBytes = bytes(input); // Convert string to bytes
array
        bytes1 firstChar;

        assembly {
            let inputPtr := add(inputBytes, 0x20) // Pointer to the first
character in the bytes array
            firstChar := mload(inputPtr)           // Load the first
character byte
        }

        return firstChar;
    }
}
```

Step-by-Step Explanation

```solidity
bytes memory inputBytes = bytes(input);
```

- Converts the input `string` to a `bytes` array.
- `inputBytes` now contains the length of the string in the first 32 bytes, followed by the string data.

```solidity
assembly {
    let inputPtr := add(inputBytes, 0x20)
}
```

- **Assembly Block**: `assembly` allows low-level manipulation of memory.
- `add(inputBytes, 0x20)`:
  - `inputBytes` is a pointer to the beginning of the array in memory.
  - The first 32 bytes store the length of the string.
  - Adding `0x20` (32 bytes in hex) skips this length prefix and points directly to the first character of the actual string data.

```solidity
firstChar := mload(inputPtr)
```

- `mload`: An assembly instruction that loads 32 bytes from a specified memory location.
- `inputPtr` points to the first character.
- However, `mload` loads 32 bytes, but the first byte of these 32 bytes is the first character of the string, while the rest are just padded zeros.

```solidity
return firstChar;
```

- Returns the extracted first character as a `bytes1` type (single byte).